RESTful 与 JSON-RPC 协议映射指南

https://gemini.google.com/app/1eb1f52bff0b4235

User prompt: RESTful 与 JSON RPC 2.0 之间的映射

Response: 将 **RESTful API** 与 **JSON-RPC 2.0** 进行映射（通常发生在 API 网关协议转换、BFF 层聚合或微服务遗留系统改造中），核心在于将 REST **散布在 HTTP 协议各层的元数据（Method、URI、Query、Header、Status Code）**，收敛并扁平化为 JSON-RPC **单一传输通道下的 JSON 报文结构（**`method`、`params`、`id`、`result`、`error`**）**。

一、 核心协议要素映射总览

RESTful 是面向资源（Resource-Oriented）**的架构风格，充分利用 HTTP 协议语义；而 JSON-RPC 2.0 是**面向过程（Procedure-Oriented）的无状态轻量级协议，通常将 HTTP 仅作为传输层管道（固定使用 `POST` 请求单一点位）。

维度	RESTful API (HTTP 语义层)	JSON-RPC 2.0 (JSON 载荷层)	映射与转换逻辑
服务端点 (Endpoint)	多路径（如 `/v1/users/101/orders`）	单一路径（如 `POST /rpc` 或 `POST /api`）	URI 路径中的资源层级转化为 `method` 的命名空间前缀
操作语义 (Action)	HTTP 动词（`GET`、`POST`、`PUT`、`PATCH`、`DELETE`）	请求体中的 `"method"` 字段	`HTTP Verb + URI 资源名` 组合映射为 `" domain.action "`
输入参数 (Inputs)	拆分为 Path 变量、Query 参数、Request Body	统一收敛至 `"params"` 字段（对象或数组）	网关需将三处数据源合并（Merge）为一个 JSON 对象
链路/请求标识	HTTP Header（如 `X-Request-ID`）	请求与响应体中的 `"id"` 字段	双向直接映射；若 JSON-RPC 省略 `"id"` 则为单向通知
成功响应	HTTP `200`/`201` + 直接返回业务 JSON	固定 HTTP `200` + `{"jsonrpc":"2.0", "result": ..., "id": ...}`	REST 响应体被包裹进 JSON-RPC 的 `"result"` 字段中
异常处理	HTTP 状态码（`4xx`/`5xx`）+ 自定义错误体	固定 HTTP `200`（传输正常）+ `"error": {"code": ..., "message": ..., "data": ...}`	HTTP 状态码映射为负整数 `error.code`，错误详情放入 `error.data`
批量操作 (Batch)	需额外设计批量接口（如 `POST /batch`）	原生支持 JSON 数组包裹多个请求对象 `[ {...}, {...} ]`	网关需将 JSON-RPC 数组拆解为并发的 REST 子请求后再聚合结果

二、 资源路径与 HTTP 动词的映射规则

在设计双向转换网关（如 Envoy JSON-RPC Transcoder 或自定义中间件）时，最通用的规范是将 REST 的 **资源名词（Noun）+ HTTP 动词（Verb）** 映射为 JSON-RPC 的 `namespace.verb`**（命名空间.动作）** 格式：

RESTful 请求形式	语义说明	JSON-RPC `method`	JSON-RPC `params` 结构建议
`GET /users`	获取资源列表（含分页/过滤）	`"users.list"`	`{"page": 1, "limit": 20, "role": "admin"}`
`GET /users/42`	获取单个指定资源	`"users.get"`	`{"id": 42}`
`POST /users`	创建新资源	`"users.create"`	`{"name": "Alice", "email": "a@example.com"}`
`PUT /users/42`	全量替换资源	`"users.replace"`	`{"id": 42, "name": "Alice", "email": "new@example.com"}`
`PATCH /users/42`	局部更新资源	`"users.update"`	`{"id": 42, "patch": {"email": "new@example.com"}}`
`DELETE /users/42`	删除指定资源	`"users.delete"`	`{"id": 42}`
`POST /users/42/orders`	创建嵌套子资源	`"users.orders.create"`	`{"user_id": 42, "items": [...]}`
`POST /users/42:lock`	非 CRUD 自定义控制器动作	`"users.lock"`	`{"id": 42, "reason": "security_audit"}`

三、 参数合并策略（Path + Query + Body → `params`）

JSON-RPC 2.0 的 `params` 支持两种格式：**按位置传参（Array）和按名称传参（Object）**。在与 REST 映射时，**强烈建议强制使用按名称传参（Named Parameters，即 JSON Object）**，以避免参数顺序耦合。

当 REST 请求转换为 JSON-RPC 时，网关存在多源参数合并问题（例如 `PATCH /orgs/{orgId}/members/{userId}?notify=true` 同时包含 Path、Query 和 Body）。主流工程解法有以下两种：

1. **命名空间隔离法（推荐，零冲突风险）：** 将不同来源的参数显式隔离在固定子键下，或者将 Body 单独包裹：

```
{
  "jsonrpc": "2.0",
  "method": "orgs.members.update",
  "params": {
    "orgId": "org_99",
    "userId": "u_42",
    "notify": true,
    "data": {
      "role": "maintainer"
    }
  },
  "id": "req-001"
}
```

2. **扁平化合并法（Flat Merge）：** 将 Path、Query、Body 的字段全部展开到 `params` 顶层对象中。若出现同名字段冲突，按固定优先级覆盖：**Path 变量优先级最高 → Request Body 次之 → Query 参数最低**（防止客户端通过 URL Query 篡改路径资源 ID）。

四、 状态码与错误码映射（Status Code ↔ Error Code）

这是两套体系最容易产生摩擦的地方：**REST 依赖 HTTP 状态码表达业务成败，而严格的 JSON-RPC 2.0 只要 HTTP 传输本身成功，即使业务报错也返回** `HTTP 200 OK`，错误通过 JSON 内部的 `error` 对象传递。

JSON-RPC 2.0 规范预留了 `-32768` 到 `-32000` 的标准错误码段，与 HTTP 状态码的标准映射关系如下：

JSON-RPC `error.code`	**JSON-RPC 官方定义**	**对应的 HTTP Status**	**典型触发场景与映射说明**
`-32700`	Parse error	`400 Bad Request`	请求体不是合法的 JSON 字符串
`-32600`	Invalid Request	`400 Bad Request`	缺少 `"jsonrpc": "2.0"` 或 `"method"` 字段类型错误
`-32601`	Method not found	`404 Not Found` / `405 Method Not Allowed`	请求的 `method` 不存在（对应 REST 路由未匹配或动词不支持）
`-32602`	Invalid params	`422 Unprocessable Entity` / `400 Bad Request`	参数校验失败（缺少必填项、字段类型不匹配）
`-32603`	Internal error	`500 Internal Server Error`	服务端内部未捕获异常
`-32001` *(自定义服务错误)*	Unauthorized	`401 Unauthorized`	Token 缺失、过期或签名无效（使用 `-32000` 至 `-32099` 预留段）
`-32003` *(自定义服务错误)*	Forbidden	`403 Forbidden`	权限不足，拒绝访问目标资源
`-32004` *(自定义服务错误)*	Resource Not Found	`404 Not Found`	**注意区分：** `-32601` 是“接口名不存在”，而查询 `user_id=999` 数据不存在应使用业务错误码（如 `-32004` 或正整数业务码）
`-32029` *(自定义服务错误)*	Too Many Requests	`429 Too Many Requests`	触发限流策略

五、 完整报文转换实例

场景：局部更新用户信息（成功与失败对比）

1. 请求报文映射：

- **RESTful 请求：**

```
PATCH /v1/users/1024?source=admin_panel HTTP/1.1
Host: api.example.com
Content-Type: application/json
X-Request-ID: 550e8400-e29b-41d4-a716-446655440000
```

```
{
  "nickname": "Neo",
  "status": "active"
}
```

- **映射后的 JSON-RPC 2.0 请求：**

```
POST /rpc HTTP/1.1
Host: api.example.com
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "method": "v1.users.update",
  "params": {
    "id": 1024,
    "source": "admin_panel",
    "body": {
      "nickname": "Neo",
      "status": "active"
    }
  },
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
```

2. 异常响应映射（如字段校验失败）：

- **RESTful 响应（HTTP 422）：**

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "error": "Validation Failed",
  "details": [{"field": "status", "issue": "must be one of [pending, active, suspended]"}]
}
```

- **映射后的 JSON-RPC 2.0 响应（HTTP 200 + Error Object）：**

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "error": {
    "code": -32602,
    "message": "Invalid params: Validation Failed",
    "data": {
      "httpStatus": 422,
      "details": [{"field": "status", "issue": "must be one of [pending, active, suspended]"}]
    }
  },
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
```

六、 特殊协议特性的映射与工程痛点

在实际架构落地时，有两个协议天然不对称的边界需要特别处理：

1. **单向通知（Notification）的映射：**
   - **JSON-RPC 特性：** 如果请求中**不包含 `"id"` 字段**，则代表这是一个 Notification（客户端不关心结果，服务端**绝不能**返回响应体）。
   - **REST 映射方案：**
     - `JSON-RPC Notification → REST`：网关向后端发起异步请求，并立即向客户端返回 `HTTP 204 No Content`（或 `202 Accepted` 空体）。
     - `REST → JSON-RPC Notification`：客户端可在 REST 请求头携带 `Prefer: respond-async` 或 `X-RPC-Notification: true`，网关据此剥离 `"id"` 字段转发给 JSON-RPC 服务端。
2. **HTTP 缓存（Caching）与幂等性退化：**
   - **问题：** REST 的 `GET` 请求天然支持浏览器、CDN 和反向代理（Nginx/Cloudflare）基于 URL 的缓存及自动重试；转换为 JSON-RPC 后，所有请求均变成 `POST /rpc`，HTTP 层彻底丧失缓存与幂等判断能力。

- **解法：** 若需保留缓存能力，可在网关层支持 **JSON-RPC over GET** 扩展规范——即对于只读方法（如 `*.get`、`*.list`），允许将 `method`、`params`（Base64URL 编码）和 `id` 放在 URL Query 中：`GET /rpc?jsonrpc=2.0&method=users.get&params=eyJpZCI6MTAyNH0=&id=1`

3. **二进制流与文件上传：**

- JSON-RPC 仅支持纯文本 JSON，若将 REST 的 `multipart/form-data` 文件直接 Base64 编码塞入 `params`，会导致体积膨胀约 **33%** 且引发网关内存峰值（OOM）。
- **解法：** 涉及大文件上传/下载时，切勿直接映射二进制流。应映射为**预签名 URL 模式（Presigned URL）**——通过 JSON-RPC 调用 `"storage.getUploadUrl"` 获取上传地址，客户端再使用标准 HTTP `PUT` 直传对象存储。

通过下方的交互式转换器，你可以切换不同的 API 操作场景，直观对比 RESTful HTTP 报文各部分是如何拆解并重组为 JSON-RPC 2.0 载荷的：

User prompt: 我要在一个嵌入式协议转换网关实现webUI，选用RESTful 还是 JSON RPC 2.0，还是其它的选项?

Response: 在**嵌入式协议转换网关**（如 Modbus/BACnet/CAN 转 MQTT/OPC-UA 网关）中实现 WebUI，行业最成熟、工程代价最低的方案不是单纯的 RESTful，而是 **“JSON-RPC 2.0（核心控制与配置）+ WebSocket/SSE（实时测点与报文监控）+ 原生 HTTP 流（固件升级与配置导入导出）”的混合架构**。

像 **OpenWrt（LuCI 的 `ubus`/`rpcd`）**、**Mongoose（Cesanta 嵌入式 Web 库）** 以及主流工业物联网网关，底层几乎全部采用 **JSON-RPC 2.0** 而非 RESTful。

一、 为什么在嵌入式网关中 JSON-RPC 2.0 往往优于 RESTful?

嵌入式网关与互联网云端后端有本质区别：它的硬件资源（RAM/Flash/连接数）受限、后端多由 C/C++/Go/Rust 编写，且业务逻辑高度偏向**设备控制与指令下发**。

评估维度	JSON-RPC 2.0	RESTful API	嵌入式网关场景下的实际影响
后端路由解析开销	**极低**（单一端点 `POST /rpc`，查表分发）	**较高**（需解析动态 URL 路径模板与 Query）	C/C++ 环境下（如 `libhv`、`Mongoose`、`esp_http_server`），写正则提取 `/ports/{id}/slaves/{sid}` 极其繁琐且易出内存错误；JSON-RPC 只需 `strcmp` 匹配 `method` 即可直接调用 C 函数指针
业务语义契合度	**天然契合**（面向动作与指令）	**较别扭**（强制抽象为名词资源 CRUD）	网关充斥大量动作型操作：串口扫描（`serial.scan`）、单次报文下发测试（`modbus.testPoll`）、重启协议栈（`service.restart`）、清空缓冲区，JSON-RPC 映射直白自然
并发连接与批量读取	**原生支持 Batch 数组请求**	**不支持**（需开启多个并发 HTTP 连接）	嵌入式 TCP 栈（如 lwIP）或轻量级 Web Server 的并发 Socket 数量极少（常限制在 4 至 8 个）。首屏加载时，JSON-RPC 可将 10 个状态查询合并为 1 个 HTTP 数组请求，避免连接池耗尽
跨传输层复用能力	**极高**（与 HTTP 解耦）	**低**（强绑定 HTTP Verb 与 URL）	网关往往不仅有本地 WebUI，还需支持**云端 MQTT 远程配置**或**本地串口上位机配置**。同一套 JSON-RPC 处理函数无需修改，即可同时服务于 HTTP、WebSocket、MQTT 和 UART
请求解析内存占用	**极低**（HTTP Header 固定且极短）	**中等**（依赖多样化的 Header 和长 URI）	减少对 HTTP 头部缓冲区的解析负担，全部逻辑收敛在 `cJSON` 或 `yyjson` 的单次解析树中

二、 为什么不能只用单一协议？（网关 WebUI 的四类流量黄金组合）

如果把网关的所有交互都硬塞进纯 HTTP 的 JSON-RPC 或 RESTful，会在**实时监控**和**固件升级**两个环节踩坑。嵌入式网关 WebUI 存在着截然不同的四类流量，推荐按以下分工组合：

1. 配置管理与控制指令 → JSON-RPC 2.0 over HTTP/WebSocket

- **涵盖业务：** 串口波特率配置、点位映射表（Tag Mapping）增删改、网络 IP 设置、重启网关、手动读写单个寄存器。
- **实现方式：** 暴露单一接口 `POST /rpc`。C 语言后端维护一张静态映射表即可完成分发：

```
typedef int (*rpc_handler_t)(cJSON *params, cJSON **result);
typedef struct {
    const char *method;
    rpc_handler_t handler;
```

```
} rpc_route_t;

static const rpc_route_t routes[] = {
    {"sys.get_info",       rpc_sys_get_info},
    {"serial.set_config", rpc_serial_set_config},
    {"modbus.write_tag",  rpc_modbus_write_tag},
};
```

2. 实时报文抓取与测点刷新 → JSON-RPC 2.0 Notifications over WebSocket（或 SSE）

- **痛点：** 协议转换网关的核心调试功能是“实时报文监视（Packet Sniffer，查看 RS485/CAN 原始十六进制收发帧）”**和**“测点实时数值刷新”。如果前端用 HTTP 定时轮询（如每 500ms 请求一次），会产生极大的 HTTP 头部解析开销并阻塞其它配置请求。
- **实现方式：**
  - **方案 A（推荐，全双工复用）：WebSocket 承载 JSON-RPC 2.0。** 前端建立一条 `/ws` 连接，既可以发送带 `"id"` 的配置请求，后端也可以主动推送**不带** `"id"` **的 JSON-RPC Notification（单向通知）**，例如实时推送抓取到的 Modbus 报文帧：

    ```
    {"jsonrpc": "2.0", "method": "sniffer.onFrame", "params": {"port": "COM1", "dir": "RX", "hex": "01 03 04 00 1A 00 2B 9
    ```

  - **方案 B（极简只读推送）：SSE（Server-Sent Events）。** 如果芯片不支持或不想引入 WebSocket 握手与帧掩码解包开销，使用原生 HTTP 长连接 `Content-Type: text/event-stream`（`GET /api/stream`）单向往前端吐 JSON 行，C 语言仅需一条 `send()` 即可实现。

3. 固件升级（OTA）与点表大文件导入导出 → 原生 HTTP 流（Multipart / Octet-Stream）

- **致命雷区： 切勿使用 JSON-RPC 传输固件文件！** 如果将 8MB 的 `.bin` 固件转为 Base64 字符串塞进 JSON-RPC 的 `"params"`，体积会膨胀至约 11MB，且 JSON 解析器（如 `cJSON`）需要将整个字符串一次性读入连续内存并分配语法树，会直接导致嵌入式设备触发 **OOM（内存耗尽）崩溃重启**。
- **实现方式：** 在 `/rpc` 之外，单独保留两个原生 HTTP 流式端点：
  - `POST /api/firmware`（接收 `application/octet-stream` 或 `multipart/form-data`，Web Server 边收 TCP 分片边按 4KB 块直接刷入 Flash 分区，内存占用恒定小于 8KB）。
  - `GET /api/config/export`（使用 HTTP Chunked 分块流式输出 CSV/JSON 点表文件）。

三、 其它候选协议评估（为什么不选它们？）

在嵌入式网关场景下，你可能还会考虑以下替代方案，但大多存在明显的工程短板：

1. **gRPC-Web / Protobuf：**
   - **不推荐用于本地 WebUI。** 浏览器无法直接发起原生 HTTP/2 gRPC 请求，必须经过 `gRPC-Web` 协议转换；且局域网通过 IP 直接访问网关时，自签名 TLS 证书与 HTTP/2 配置极易被浏览器拦截。此外，WebUI 开发者在浏览器 F12 调试面板中无法直接阅读二进制报文，严重拖慢联调效率。
2. **CBOR-RPC / MessagePack-RPC：**
   - **仅在极端受限的 MCU 上考虑。** 将 JSON 替换为二进制编码（CBOR 或 MessagePack），可省去 ASCII 字符串转浮点数的 CPU 开销。但除非你的网关运行在 RAM 小于 128KB 的裸机/RTOS 单片机上且高频传输上千个浮点测点，否则现代轻量级 JSON 库（如 `cJSON` 或 `yyjson`）的速度完全够用，没必要牺牲前后端可读性。
3. **GraphQL：**
   - **完全不适合嵌入式。** 在嵌入式端实现 AST 查询解析器的内存和算力开销过大，属于过度设计。

四、 不同硬件档次的最终选型建议

你可以根据当前网关的**硬件规格与后端语言**直接对号入座：

网关硬件平台档次	典型芯片与系统	后端语言与 Web 库	最佳协议选型方案
轻量级 MCU 网关	ESP32-S3 / STM32H7 (FreeRTOS + lwIP，RAM 小于 512KB)	C + `Mongoose` / `esp_http_server`	**WebSocket 承载 JSON-RPC 2.0（合二为一）** + 独立 `POST /ota` 裸流接口。单条长连接搞定全部控制与实时推送，最大限度节省 Socket 描述符
标准嵌入式 Linux	i.MX6ULL / 全志 T113 / MT7628 (OpenWrt / Buildroot，RAM 64MB	C/C++ (`libhv` / `uhttpd` / `CivetWeb`)	`POST /rpc` **(JSON-RPC 2.0)** 处理配置与控制 + `GET /events` **(SSE)** 推送实时报文与测点 + 原生 HTTP 流处理固件升级

网关硬件平台档次	典型芯片与系统	后端语言与 Web 库	最佳协议选型方案
网关	至 256MB)		
高性能边缘计算网关	RK3568 / RK3588 / x86 (Debian/Ubuntu，RAM 大于 512MB)	Go (`Gin`/`Echo`) 或 Rust (`Axum`)	若仅服务自有 WebUI 且需复用至 MQTT 云管，选 **JSON-RPC 2.0**；若客户要求提供标准第三方 IT 系统对接文档（Swagger/OpenAPI），选 **RESTful + WebSocket**

通过下方的交互式选型评估器，你可以根据网关的**硬件资源限制**、**实时抓包监控频率**以及**云端/串口多通道复用需求**调整权重，查看不同协议架构在各项工程指标上的表现对比：
